TIP102 | Intermediate Technical Interview Prep

Intermediate Technical Interview Prep Fall 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **134071**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Getting Started

Learning with AI ✨

▶ Course Info

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Week 6: Session 1

Recursion I & Linked Lists III

In this session, students will deepen their ability to work with linked lists, solving problems involving traversal, node manipulation, and cycle detection. These exercises will cover techniques common to many linked lists problems and enhance their understanding of this crucial data structure.

> You can find session recordings and more on the resources tab. Session slide decks are available on the overview tab.

📈 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👷 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like VSCode, PyCharm, Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

Problem Set Version 1

▼ Problem 1: Building a Playlist

The assignment statement to the `top_hits_2010s` variable below creates the linked list `Uptown Funk -> Party Rock Anthem -> Bad Romance`. Break apart the assignment statement into multiple lines with one call to the `Node` constructor per line to recreate the list.

```
class SongNode:
    def __init__(self, song, next=None)
        self.song = song
        self.next = next

# For testing
def print_linked_list(node):
    current = node
    while current:
        print(current.song, end=" -> " if current.next else "")
        current = current.next
    print()

top_hits_2010s = SongNode("Uptown Funk", SongNode("Party Rock Anthem", SongNode("Bad Romar
```

Example Usage:

```
print_linked_list(top_hits_2010s)
```

Example Output:

```
Uptown Funk -> Party Rock Anthem -> Bad Romance
```

▶ 💡 **Hint: Nested Constructors**

Close Section

▼ Problem 2: Top Artists

Given the head of a linked list `playlist`, return a dictionary that maps each *artist* in the list to its frequency.

Evaluate the time complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time and space complexity.

```
class SongNode:
    def __init__(self, song, artist, next=None)
        self.song = song
        self.artist = artist
        self.next = next

# For testing
def print_linked_list(node):
    current = node
    while current:
        print((current.song, current.artist), end=" -> " if current.next else "")
        current = current.next
    print()


def get_artist_frequency(playlist):
    pass
```

Example Usage:

```
playlist = SongNode("Saturn", "SZA",
                SongNode("Who", "Jimin",
                        SongNode("Espresso", "Sabrina Carpenter",
                                SongNode("Snooze", "SZA"))))

get_artist_frequency(playlist)
```

Example Output:

```
{ "SZA": 2, "Jimin" : 1, "Sabrina Carpenter": 1}
```

Close Section

▼ Problem 3: Glitching Out

The following code attempts to remove the first node with a given `song` from a singly linked list with head `playlist_head` but it contains a bug!

Step 1: Copy this code into your IDE.

Step 2: Create your own test cases to run the code against, and use print statements and the stack trace to identify and fix the bug so that the function correctly removes a node by value from the list.

Step 3: Evaluate the time and space complexity of the fixed solution. Define your variables and provide a rationale for why you believe the solution has the stated time and space complexity.

```
class SongNode:
    def __init__(self, song, artist, next=None)
        self.song = song
        self.artist = artist
        self.next = next

# For testing
def print_linked_list(node):
    current = node
    while current:
        print((current.song, current.artist), end=" -> " if current.next else "")
        current = current.next
    print()


# Function with a bug!
def remove_song(playlist_head, song):
    if not playlist_head:
        return None
    if playlist_head.song == song:
        return playlist_head.next

    current = playlist_head
    while current.next:
        if current.next.song == song:
            current = current.next.next
            return playlist_head
        current = current.next

    return playlist_head
```

Example Usage:

```
playlist = SongNode("SOS", "ABBA",
                SongNode("Simple Twist of Fate", "Bob Dylan",
                    SongNode("Dreams", "Fleetwood Mac",
                        SongNode("Lovely Day", "Bill Withers"))))

print_linked_list(remove_song(playlist, "Dreams"))
```

Expected Output:

```
('SOS', 'ABBA') -> ('Simple Twist of Fate', 'Bob Dylan') -> ('Lovely Day', 'Bill Withers')
```

Close Section

▼ Problem 4: On Repeat

A variation of the two-pointer technique introduced in previous units is to have a slow and a fast pointer that increment at different rates.

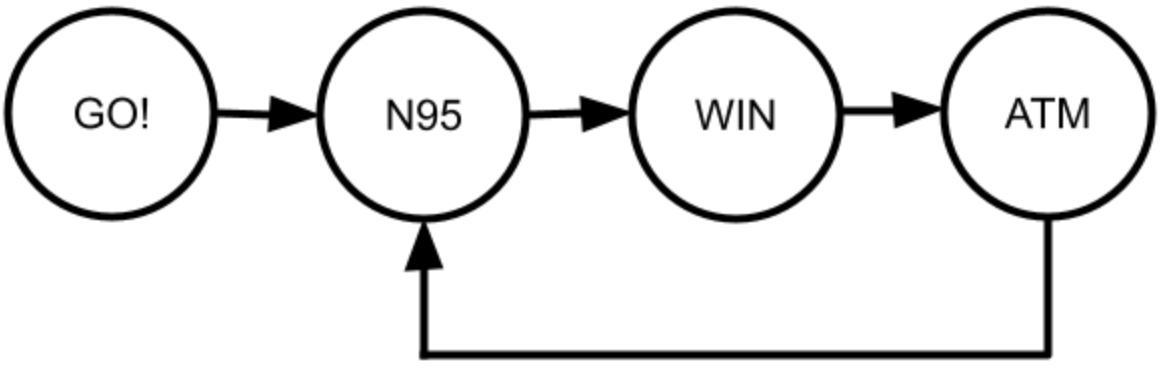
We would like to check whether our playlist loops or not. Given the head of a linked list `playlist_head`, return `True` if the playlist has a cycle in it and `False` otherwise. A linked list has a cycle if at some point in the list, the node's next pointer points back to a previous node in the list.

Evaluate the time and space complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time and space complexity.

```
class SongNode:
    def __init__(self, song, artist, next=None)
        self.song = song
        self.artist = artist
        self.next = next

def on_repeat(playlist_head):
    pass
```

Example Usage:




```
song1 = SongNode("GO!", "Common")
song2 = SongNode("N95", "Kendrick Lamar")
```

```
song3 = SongNode("WIN", "Jay Rock")
song4 = SongNode("ATM", "J. Cole")
song1.next = song2
song2.next = song3
song3.next = song4
song4.next = song2

print(on_repeat(song1))
```

Example Output:

```
True
```

▶ ✨ AI Hint: Slow and Fast Pointers

Close Section

▼ Problem 5: Looped

Given the head of a linked list `playlist_head` that may contain a cycle, use the fast and slow pointer method to return the length of the cycle. If the list does not contain a cycle, return `0`.

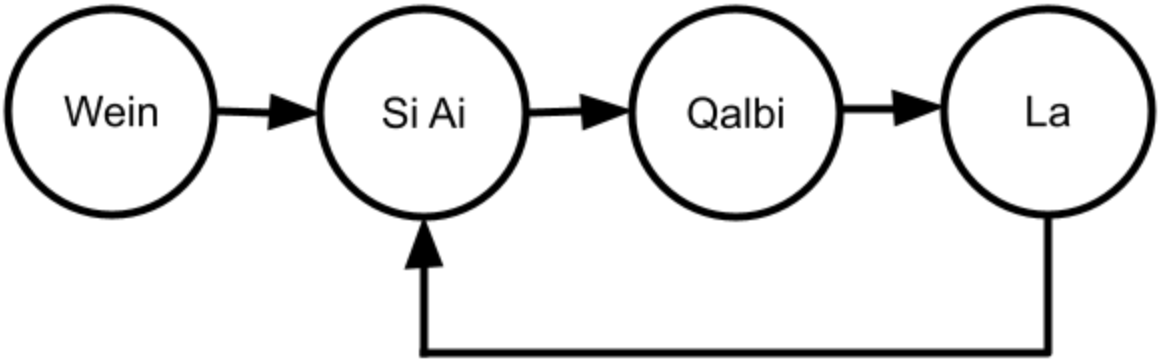
Evaluate the time and space complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time and space complexity.

```
class SongNode:
    def __init__(self, song, artist, next=None)
        self.song = song
        self.artist = artist
        self.next = next

# For testing
def print_linked_list(node):
    current = node
    while current:
        print((current.song, current.artist), end=" -> " if current.next else "")
        current = current.next
    print()

def loop_length(playlist_head):
    pass
```

Example Usage:




```
song1 = SongNode("Wein", "AL SHAMI")
song2 = SongNode("Si Ai", "Tayna")
song3 = SongNode("Qalbi", "Yasser Abd Alwahab")
song4 = SongNode("La", "DYSTINCT")
song1.next = song2
song2.next = song3
song3.next = song4
song4.next = song2

print(loop_length(song1))
```

Example Output:

```
3
```

▶ ✨ AI Hint: Multiple Pass Technique

Close Section

▼ Problem 6: Volume Control

You are working as an engineer normalizing volume levels on songs. Given the head of a singly linked list with integer values `song_audio` representing volume levels at different points in a song, return the number of critical points. A critical point is a local minima or maxima.

- The head and tail nodes are not considered critical points.
- A node is a local minima if both the next and previous elements are greater than the current element
- A node is a local maxima if both the next and previous elements are less than the current element

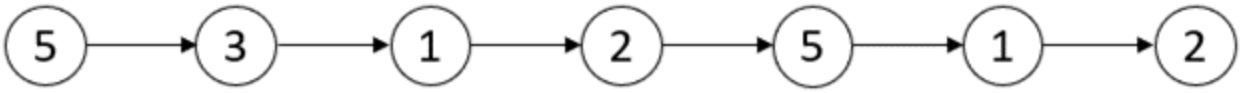
Evaluate the time and space complexity of your solution. Define your variables and provide a rationale for why you believe your solution has the stated time and space complexity.

```python
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def count_critical_points(song_audio):
    pass
```

Example Usage:




```python
song_audio = Node(5, Node(3, Node(1, Node(2, Node(5, Node(1, Node(2)))))))

print(count_critical_points(song_audio))
```

Example Output:

```
3
Explanation: There are three critical points:
- The third node is a local minima because 1 is less than 3 and 2.
- The fifth node is a local maxima because 5 is greater than 2 and 1.
- The sixth node is a local minima because 1 is less than 5 and 2.
```

▶ 💡 **Hint: Which technique?**

Close Section

Problem Set Version 2

- ▶ Problem 1: Why is it Always You Three
- ▶ Problem 2: 200 Points for Gryffindor
- ▶ Problem 3: Target Practice
- ▶ Problem 4: Turn Back Time
- ▶ Problem 5: Mirror, Mirror
- ▶ Problem 6: Magic Loop